Neuronauts Model & Architecture Note

Captured from design discussion. Covers: agent/grammar/GAT pipeline, global vs box-scale, and the question of hierarchical assembly.

Architecture Overview

```
EM volume + CAVE synapses
        |
        v
1. Perception          fetch.py · vectorized.py · membrane_unet.py
   700 agents × 450 steps → path_arr + synapse_hits
   (Membrane field drives agent traversal; agents walk through tissue)

        v
2. Scaffold init       run.py → _scaffold_union_from_seg_ids
   CAVE seg-IDs pre-group same-supervoxel agents → ~10× reduction

        v
3. Shared Grammar      grammar.py · shared_grammar_model.py
   TorchPathEncoder, MergeScorer, BridgeHead
   Learns over paths; does NOT run agents or membranes

        v
4. Global Assembly     assembly.py · shared_grammar_model.py
   GlobalAssemblyGAT: sparse attention over ConnectivityGraph
   Refines edges; does NOT do agent traversal

        v
5. Evaluation          line_graph.py
   Synapse line-graph F1
```

What Is an Agent?

An **agent** is one of the 700 virtual walkers that trace paths through the EM volume.

- **Agent** = one path/trace. Each has: `agent_id` (0–699), `path` (sequence of 3D voxel points), `visited_synapses`.
- Agents walk in the membrane field (repelled by membranes, attracted to synapses).
- One agent ~ one neurite trace. Many agents can represent one neuron after merging.

Pipeline mapping:

1. 700 agents → `path_arr [700, 450, 3]`, `synapse_hits [700, n_syn]`
2. Scaffold init groups agents by CAVE seg-ID
3. Merge + beam search: Grammar MergeScorer decides which agent-pairs to merge
4. **MergedNeuron** = 1+ agents merged together. Has `path_points`, `synapse_indices`, `agent_ids`
5. **ConnectivityGraph** = nodes = MergedNeurons, edges = synapse connections
6. **GAT** sees MergedNeurons (not raw agents). Node features = path_encoder(path_points). Refines which synapse edges to keep.

The GAT does not run agents. It operates on the graph of fragments and synapse edges after merge.

Grammar vs GAT

Component	Role	Uses EM/membranes?	Uses agents?
Perception	Membrane field + agent traversal	Yes	Yes
Grammar	Path encoding, merge scoring, atomicity, bridge	No (paths derived from agents)	Indirect (paths from agents)
GAT	Global graph refinement	No	No (sees MergedNeurons)

Grammar training can run **without agent simulation** — it uses cached synapse tables for merge/atomicity. GAT training needs agent simulation to build ConnectivityGraphs.

Global vs Box-Scale

Current pipeline: Box-level (6–30 µm). One box → one ConnectivityGraph → one GAT pass.

Minnie65 scale: ~120k neurons, ~300M synapses, ~1 mm^3. "All at once" is infeasible.

Decomposition options:

1. **Per-nucleus** — One subgraph per neuron (soma position + synapse cloud). Embarrassingly parallel.
2. **Tiling** — Overlapping 20–40 µm tiles, run agents + merge per tile, stitch at boundaries.
3. **Soma graph** — Neuron × neuron graph (120k nodes, sparse edges). GAT over neurons, not fragments. See `experiments/soma_graph/`.

Do We Need a Hierarchical Global Assembly Step?

Short answer: Not for the current box-level pipeline. Yes if we want whole-dataset inference.

Current design: GAT is "global" only within a single box — it sees all fragments and edges in that box. For 50–200 nodes per box, a single GPU is sufficient.

Hierarchical step would mean:

- **Level 1 (current):** Box-level merge + GAT → refined ConnectivityGraph per box
- **Level 2 (proposed):** Cross-box stitching / refinement
  - Option A: Fuse overlapping box boundaries (same fragments appear in multiple tiles)
  - Option B: Soma-level graph — aggregate box outputs into neuron × neuron, run GAT at that scale
  - Option C: Coarse-to-fine — cheap heuristics on 100 µm to propose clusters, refine with agents+GAT on smaller subvolumes

When to add it: When we move beyond single-box evaluation (e.g. soma graph experiment, multi-box validation, or full Minnie65 inference). The current pipeline is complete for box-scale; hierarchical assembly is the bridge to global.

Iterative Optimization (Codex/Gemini)

- `scripts/codex_optimize.py` — Patch → evaluate → keep/revert loop
- `program.md` — Mission + architecture; context for the LLM
- **Default target:** `neuronauts/grammar.py`

- **Validation:** Research cycle → val_f1; keep changes that improve it
- **Backends:** Codex, Claude, Gemini

The LLM modifies code to improve validation F1 without hard-coded heuristics. The learned object is the grammar/GAT; the LLM is an outer optimizer.

Morphology vs Connectivity

Current supervision: Root IDs (same-root = same neuron). Implicit structure, no explicit morphology model.

Possible addition: Morphology / validity head trained on CAVE segments — "does this cluster look like a valid neuron?" Could use pooled path features per root as a prior or diagnostic.

Program.md: Richer path features (skeleton tortuosity, mesh volume-surface ratio) noted as future improvement.

Key Files

File	Purpose
`neuronauts/vectorized.py`	Agent simulation
`neuronauts/merge.py`	MergedNeuron, merge_agents
`neuronauts/run.py`	Full pipeline, _build_graph
`neuronauts/assembly.py`	gat_refine_connectivity, label_graph_edges
`neuronauts/shared_grammar_model.py`	GlobalAssemblyGAT, gat_train_step
`experiments/soma_graph/`	Soma-level neuron × neuron experiment

Generated from design discussion. Last updated: 2025-03.